АЙСМАРТ

Алгоритм и программа - как описывать решение задачи

Занятие 5. Алгоритм и программа — как описывать решение задачи

Содержание

- 1. Актуализация знаний
- 2. Вступление
- 3. Теоретический материал 3.1. Что такое алгоритм 3.2. Исполнитель алгоритма 3.3. Входные и выходные данные 3.4. Основные алгоритмические конструкции 3.5. Блок-схемы 3.6. От алгоритма к программе
- 4. Практические кейсы
- 5. Задания для самопроверки
- Итог занятия
- Глоссарий
- Материалы к занятию

1. Актуализация знаний

Ответьте на вопросы, чтобы вспомнить материал прошлых занятий. Свой ответ можно сверить, раскрыв строку «Показать ответ».

Вопрос 1.

Как называется устройство или объект, выполняющий команды алгоритма?

- А) Переменная
- Б) Исполнитель
- В) Условие
- Г) Программа

Показать ответ

Б) Исполнитель

Вопрос 2.

Что такое входные данные?

- А) Результат работы программы
- Б) Исходная информация, поступающая для решения задачи
- В) Ошибки в программе
- Г) Названия переменных

Показать ответ

Б) Исходная информация, поступающая для решения задачи

Вопрос 3.

Какой геометрической фигурой обозначается условие в блок-схеме?

- А) Овал
- Б) Прямоугольник
- В) Ромб
- Г) Стрелка

Показать ответ

В) Ромб

Вопрос 4.

Как называется алгоритмическая конструкция, при которой действия выполняются несколько раз?

- А) Последовательность
- Б) Ветвление
- В) Повторение (цикл)
- Г) Условие

Показать ответ

В) Повторение (цикл)

Вопрос 5.

Что выведет программа?

```
x = 10
if x > 5:
 print('Больше')
else:
 print('Меньше')
```

- А) Больше
- Б) Меньше

- В) 10
- Г) Ошибка

Показать ответ

А) Больше

2. Вступление

Каждый день вы сталкиваетесь с алгоритмами, даже не замечая этого. Приготовление бутерброда, переход через дорогу по светофору, использование банкомата или поиск маршрута в навигаторе – всё это примеры выполнения определённой последовательности действий.

Однако простой перечень действий не всегда приводит к нужному результату. Команды должны быть понятными, располагаться в правильном порядке и учитывать возможные условия. Например, при переходе через дорогу действие зависит от сигнала светофора, а при поиске маршрута некоторые шаги повторяются до достижения пункта назначения.

Компьютеры не умеют самостоятельно понимать человеческие желания. Для выполнения любой задачи им необходимо получить точное описание всех действий. Такое описание называется

алгоритмом.

На этом занятии вы узнаете: что такое алгоритм, кто такой исполнитель, как задаются входные и выходные данные, зачем нужны условия и повторения, а также каким образом алгоритм превращается в программу.

После этого занятия вы сможете:

- объяснять понятие алгоритма и называть его свойства;
- различать входные и выходные данные;
- строить алгоритмы для решения простых задач;
- читать и составлять блок-схемы;
- переходить от алгоритма к программному коду.

При изучении материала обращайте внимание не только на отдельные команды, но и на связи между ними: какие данные получает алгоритм, какие действия выполняет и какой результат должен получить исполнитель.

3. Теоретический материал

3.1. Что такое алгоритм

Алгоритм – это точное описание последовательности действий, приводящих к решению задачи за конечное число шагов.

Алгоритм можно сравнить с инструкцией: исполнитель последовательно выполняет указанные действия и получает требуемый результат. При этом описание должно быть достаточно точным,

чтобы исполнитель не был вынужден самостоятельно догадываться, что именно от него требуется.

Одна и та же задача может решаться разными алгоритмами. Например, вычисления можно выполнить вручную, с помощью калькулятора или программы. Алгоритмы будут отличаться набором команд, но должны приводить к правильному результату.

Свойства алгоритма:

Свойство	Описание
Дискретность	Алгоритм разбит на отдельные шаги
Точность	Каждый шаг описан однозначно
Конечность	Алгоритм завершается за конечное число шагов
Результативность	Алгоритм даёт результат
Массовость	Алгоритм применим к классу однотипных задач

Дискретность означает, что выполнение можно представить как цепочку отдельных команд. Точность не допускает двусмысленных формулировок: команда «немного подождать» менее точна, чем команда «подождать 3 минуты».

Конечность требует, чтобы выполнение завершалось, а не продолжалось бесконечно. Результативность означает получение результата, соответствующего поставленной задаче. Массовость проявляется, например, в алгоритме вычисления площади прямоугольника: его можно применять к разным значениям длины и ширины, а не только к одной паре чисел.

Пример. Задача: приготовить чай.

- Взять чайник.
- Налить воду.
- Включить чайник.
- Дождаться кипения.
- Положить пакетик чая в кружку.
- Налить кипяток.
- Подождать 3 минуты.
- Чай готов.

В этом примере важен порядок действий. Нельзя налить кипяток до того, как вода закипела. Это показывает, что перестановка шагов алгоритма может изменить результат или сделать выполнение невозможным.

Обратите внимание:

не всякий список действий является правильным алгоритмом. В нём должны быть понятные

команды, определённый порядок выполнения и условие завершения.

Например, команда «выполнять действие бесконечно» без предусмотренной остановки нарушает свойство конечности: алгоритм никогда не завершится.

3.2. Исполнитель алгоритма

Исполнитель – объект, который способен выполнять команды алгоритма. Исполнитель понимает только те команды, которые входят в его систему команд.

Система команд исполнителя

– это полный набор команд, которые данный исполнитель умеет распознавать и выполнять. Поэтому один и тот же алгоритм может быть понятен человеку, но непригоден для робота или компьютера без дополнительного уточнения команд.

Исполнитель	Примеры команд
Человек	Ходить, писать, считать
Робот	Перемещаться, брать предметы
Компьютер	Выполнять программы, хранить данные
Умная колонка	Воспроизводить музыку, отвечать на вопросы

Например, команда «сделать шаг вперёд» может входить в систему команд робота, но команда «пойти куда-нибудь» для него недостаточно точна: не указаны направление и расстояние.

Важно: исполнитель способен выполнять только те команды, которые входят в его систему команд. Калькулятор не может приготовить чай.

Кроме того, команда должна быть записана в понятной исполнителю форме. Компьютер не выполняет словесную инструкцию напрямую: её необходимо представить в виде программы на поддерживаемом языке программирования.

Частая ошибка – путать исполнителя и алгоритм. Исполнитель выполняет действия, а алгоритм описывает, какие именно действия и в каком порядке необходимо выполнить.

3.3. Входные и выходные данные

Чтобы решить задачу, алгоритму часто требуется исходная информация. После её обработки появляется результат. Поэтому при постановке задачи важно заранее определить, какие данные поступают на вход и что должно быть получено на выходе.

Тип	Описание	Пример
Входные данные	Информация, поступающая для решения задачи	Длина и ширина прямоугольника

Выходные данные	Результат работы алгоритма	Площадь прямоугольника

Пример. Задача: найти сумму двух чисел.

Входные данные: A = 5, B = 3

Выходные данные: S = 8

Алгоритм обработки этих данных состоит из действия сложения: S = A + B. Если изменить входные значения, последовательность действий останется прежней, но выходные данные изменятся.

Входные данные не обязательно являются числами. Ими могут быть текст, ответы пользователя, изображения или логические значения. Выходом также может быть не только число, но и сообщение, решение или другой обработанный результат.

Важно:

переменные, используемые для хранения промежуточных вычислений, не следует автоматически считать входными или выходными данными. Вход поступает в алгоритм извне, а выход представляет его итоговый результат.

Например, при переводе температуры из градусов Цельсия в градусы Фаренгейта температура в градусах Цельсия – входное значение, а температура в градусах Фаренгейта – выходное.

3.4. Основные алгоритмические конструкции

Любой алгоритм строится из трёх базовых конструкций. Они позволяют описать линейное выполнение команд, выбор одного из вариантов и многократное повторение действий.

Конструкция	Описание	Пример
Последовательность	Действия выполняются строго друг за другом	Шаги приготовления чая
Ветвление (условие)	Выбор действия зависит от проверки условия	Если дождь ☒ взять зонт, иначе ☒ выйти без зонта
Повторение (цикл)	Некоторые действия выполняются несколько раз	Пока не решены все задачи ☒ решать следующую

В реальном алгоритме конструкции могут сочетаться. Например, внутри цикла может находиться условие, а в каждой ветви условия – последовательность из нескольких команд.

Ветвление требует условия, результатом проверки которого является один из двух вариантов: истина или ложь. В зависимости от результата исполнитель выбирает соответствующую ветвь и не выполняет команды другой ветви.

Цикл должен содержать понятное правило повторения. Если условие завершения никогда не станет истинным или ложным в нужный момент, выполнение может продолжаться бесконечно.

Пример алгоритма для ИИ-системы:

Задача: подготовить данные для оценки знаний ученика.

- Получить ответы ученика на задания.
- Сравнить каждый ответ с заданным критерием проверки.
- Зафиксировать результат проверки каждого задания.
- Подсчитать количество выполненных заданий.
- Передать подготовленные результаты для оценки знаний.

В этом алгоритме ответы ученика являются входными данными, а подготовленные результаты проверки – выходными. Проверка нескольких заданий может быть организована как повторение одних и тех же действий для каждого ответа.

Пример ветвления:

```
Если идёт дождь,
 то взять зонт,
 иначе выйти без зонта.
```

Здесь условием является высказывание «идёт дождь». Если оно истинно, выполняется первая команда; если ложно – команда после слова «иначе».

Пример повторения:

```
Пока не решены все задачи:
 прочитать задачу;
 решить задачу;
 проверить ответ.
```

Одна итерация такого цикла включает чтение, решение и проверку одной задачи. После каждой итерации необходимо перейти к следующей нерешённой задаче. Когда нерешённых задач не останется, цикл завершится.

Частая ошибка:

условие и цикл выполняют разные функции. Условие выбирает ветвь действий, а цикл организует повторение. При этом условие может использоваться для определения момента завершения цикла.

3.5. Блок-схемы

Блок-схема – графическое представление алгоритма. Позволяет наглядно описать логику решения задачи.

Фигуры блок-схемы соединяются стрелками. Стрелки показывают, в каком порядке исполнитель

переходит от одного шага к другому. Чтение обычно начинается с блока «Начало» и заканчивается блоком «Конец».

Фигура	Название	Назначение
Овал	Начало / Конец	Обозначает начало и конец алгоритма
Прямоугольник	Действие	Вычисление, присваивание, вывод
Ромб	Условие	Ветвление: проверка условия (Да/Нет)
Параллелограмм	Ввод / Вывод	Получение или вывод данных
Стрелка	Переход	Направление выполнения алгоритма

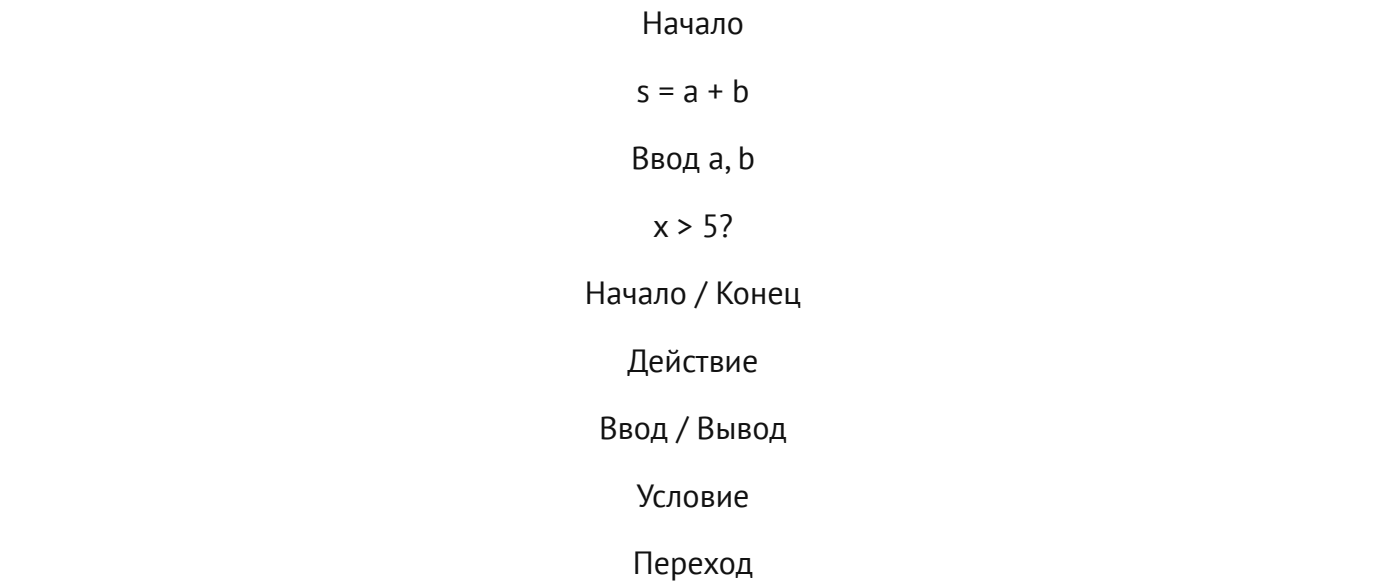
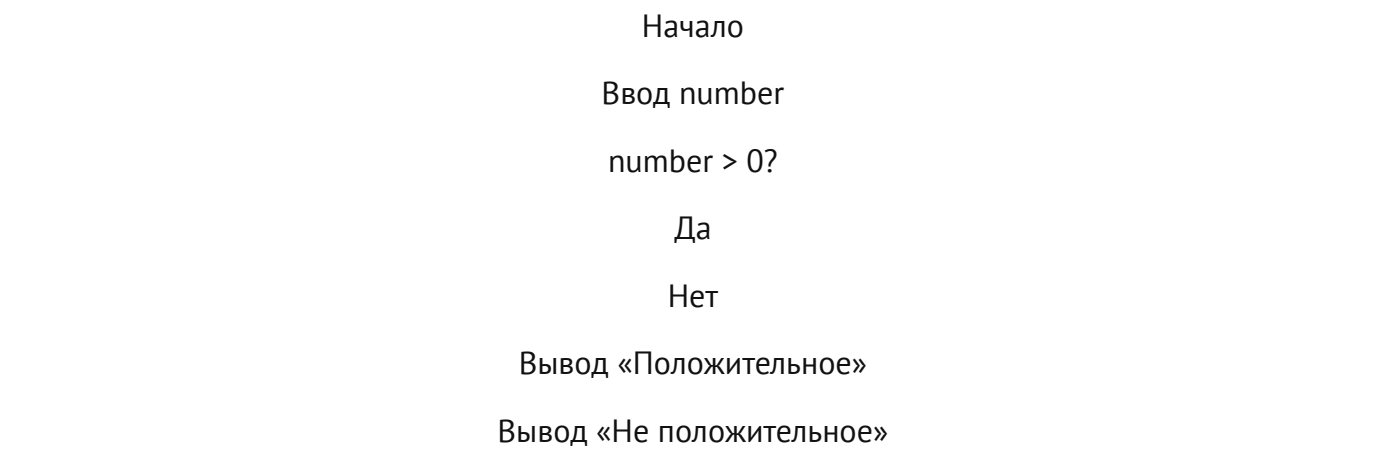


Рисунок 1. Основные фигуры блок-схемы

Например, блок-схема определения знака числа может включать ввод числа, проверку условия «число больше нуля?» и два варианта вывода. При ответе «Да» выводится сообщение о положительном числе, а при ответе «Нет» выполняется другая ветвь.



Конец

Рисунок 2. Блок-схема определения положительного числа

Следует учитывать граничные значения. Если проверяется только условие «число больше нуля», то ноль попадёт в ветвь «Нет» вместе с отрицательными числами. Это не означает, что ноль является отрицательным: ветвь лишь сообщает, что исходное условие оказалось ложным.

Типичные ошибки при составлении блок-схем:

- неверный выбор геометрической фигуры;
- отсутствие направляющих стрелок;
- нет обозначений Да/Нет у ромба;
- обрыв одной из ветвей без перехода к следующему блоку или завершению алгоритма;
- запись действия в ромбе вместо логического условия;
- отсутствие блоков начала и конца.

Обратите внимание:

в ромбе записывают вопрос или условие, которое можно оценить как истинное или ложное. Например: «x > 5?». Команду «увеличить x на 1» следует помещать в прямоугольник.

Например, команду «ввести значение длины» изображают параллелограммом (ввод данных), а вычисление площади – прямоугольником (действие).

3.6. От алгоритма к программе

Программа – это алгоритм, записанный на языке программирования. Каждая конструкция алгоритма соответствует конструкции языка.

Перед написанием кода полезно определить входные данные, ожидаемый результат и порядок действий. Это уменьшает вероятность пропустить важную проверку или неправильно расположить команды.

Конструкция алгоритма	Python
Последовательность	Строки кода выполняются сверху вниз
Ветвление	if … else …
Повторение	while … / for …
Ввод данных	input()
Вывод данных	print()

Функция input() получает введённые пользователем данные как строку. Если требуется выполнять арифметические действия, данные преобразуют в числовой тип, например с помощью int() для целого числа или float() для числа с дробной частью.

Пример. Определить, является ли число положительным. Это та же задача, что на рисунке 2, записанная на Python:

```
number = int(input('Введите число: '))
if number > 0:
 print('Положительное')
else:
 print('Не положительное')
```

Разбор кода:

- number = int(input(...)) – ввод числа и преобразование в целое;
- if number > 0: – проверка условия;
- print() – вывод результата.

Если пользователь введёт 7, условие number > 0 будет истинным и программа выберет первую ветвь. Если будет введено 0 или отрицательное число, условие окажется ложным и будет выбрана ветвь else.

Важно:

сообщение «Не положительное» объединяет два случая: отрицательное число и ноль. Если их необходимо различать, потребуется дополнительная проверка.

В Python тело каждой ветви выделяется отступом (обычно 4 пробела): строки print находятся внутри соответствующих блоков if и else.

Пример. Расчёт скидки клиента:

```
purchase_amount = float(input('Введите сумму покупки: '))
if purchase_amount > 10000:
 discount = purchase_amount * 0.10
else:
 discount = 0
print('Скидка:', discount, 'руб.')
```

Входными данными здесь является сумма покупки purchase_amount, а выходными – рассчитанное значение discount. Если сумма строго больше 10000, скидка составляет 10% от суммы покупки. В противном случае переменной discount присваивается 0.

Граничный случай – покупка ровно на 10000 рублей. Условие purchase_amount > 10000 для неё ложно, поэтому скидка не начисляется. Если по условию задачи скидка должна действовать начиная с 10000 рублей, знак сравнения необходимо выбирать с учётом этого требования.

Такой алгоритм может использоваться в кассовой системе магазина.

Частая ошибка:

знак = используется для присваивания значения переменной, а сравнение на равенство в условии записывается с помощью ==. Также после условия if в Python обязательно ставится двоеточие.

4. Практические кейсы

Алгоритмы работают в привычных нам устройствах и сервисах. В каждом кейсе можно выделить входные данные, шаги алгоритма и результат.

Кейс 1. Навигатор строит маршрут

Входные данные:

начальная точка, конечная точка.

Алгоритм:

1. Получить начальную и конечную точки.
2. Проанализировать все доступные дороги.
3. Найти возможные маршруты.
4. Выбрать оптимальный путь (по времени или расстоянию).

Выходные данные:

маршрут движения.

Кейс 2. Банкомат выдаёт деньги

Входные данные:

карта, PIN-код, сумма снятия.

Алгоритм:

1. Прочитать карту.
2. Проверить PIN-код.
3. Проверить остаток на счёте.
4. Если баланс достаточен ☒ выдать деньги и списать сумму.
5. Иначе ☒ вывести сообщение об ошибке.

Выходные данные:

деньги или сообщение об ошибке.

Во втором кейсе используется ветвление: результат зависит от проверки условия «баланс достаточен». В первом кейсе поиск среди многих дорог и маршрутов — это повторение одних и тех же действий для каждого варианта.

5. Задания для самопроверки

Выполните задания самостоятельно. Задания с вариантами ответа решайте без запуска кода. В заданиях с подсказками сначала попробуйте справиться без них.

Задание 1. Алгоритм приготовления бутерброда

Условие

Составьте алгоритм приготовления бутерброда из 6–8 шагов.

Что нужно сделать

Запишите шаги в повелительной форме.

Проверьте: можно ли поменять шаги местами без потери результата?

При проверке алгоритма представьте, что исполнитель не знает предполагаемой последовательности заранее. Каждая команда должна быть понятной и выполнимой.

Задание 2

Условие

Определите входные и выходные данные для задачи:

«Вычислить площадь прямоугольника».

Что нужно сделать

Запишите в виде:

Входные данные: …

Выходные данные: …

Дополнительно укажите действие, которое связывает входные данные с результатом.

Критерии самопроверки базового уровня:

- шаги записаны в правильном порядке;
- каждая команда сформулирована однозначно;
- входные данные не перепутаны с результатом;
- после выполнения алгоритма задача считается решённой.

Задание 3

Условие

Выполните алгоритм вручную для числа 10 и определите результат:

1. Ввести число.

2. Умножить на 2.

3. Прибавить 5.

4. Вывести результат.

А) 20 Б) 25 В) 15 Г) 17

Подсказка

Для проверки записывайте значение после каждого шага. Это поможет не пропустить изменение числа.

Задание 4

Условие

Прочитайте код. Не запуская его, определите результат:

Исходный код

```
number = 8
if number > 5:
 print('Да')
else:
 print('Нет')
```

Что нужно сделать

А) Да Б) Нет В) 8 Г) Ошибка

Подсказка

Сначала определите значение логического выражения number > 5, а затем выберите выполняемую ветвь.

Задание 5

Условие

В программе допущена одна ошибка. Найдите её и исправьте:

Исходный код

```
number = int(input())
if number > 0
 print('Положительное')
```

Что нужно сделать

Запишите исправленную программу и объясните, в чём была ошибка.

Подсказка

Что обязательно ставится после условия в конструкции

if

в Python?

Задание 6. Алгоритм определения чётности числа

Условие

Напишите алгоритм (словесно или в виде блок-схемы), который определяет, является ли введённое целое число чётным или нечётным.

Это нужно знать

Число чётное, если оно делится на 2 без остатка. Остаток от деления в Python вычисляет оператор

```
print(8 % 2) # 0 — остатка нет, 8 чётное
print(7 % 2) # 1 — остаток 1, 7 нечётное
```

Значит, условие чётности числа

можно записать так:

n % 2 == 0

Что нужно сделать

Запишите алгоритм, а затем программу на Python.

При составлении условия используйте остаток от деления числа на 2. Продумайте два возможных результата проверки.

Проверьте алгоритм на нескольких значениях, включая 0, положительное и отрицательное целое число.

Задание 7

Условие

Разработайте алгоритм автоматической проверки допуска студента к экзамену и запишите его программой на Python.

Условия допуска

- посещаемость не менее 70%;
- выполнены все практические работы.

Учтите, что допуск возможен только при одновременном выполнении обоих требований.

Проверьте алгоритм как минимум для трёх случаев: выполнены оба требования, выполнено только одно требование, не выполнено ни одно требование.

Отдельно проверьте граничное значение посещаемости 70%: формулировка «не менее 70%» означает, что это значение удовлетворяет условию.

Итог занятия

На занятии рассмотрены:

- понятие алгоритма и его свойства;
- исполнитель и система команд;
- входные и выходные данные;
- три базовые алгоритмические конструкции: последовательность, ветвление, повторение;
- блок-схемы и их элементы;
- переход от алгоритма к программному коду на Python;
- примеры алгоритмов в реальных системах: навигатор и банкомат.

При решении новой задачи полезно действовать по общему плану:

- сформулировать требуемый результат;
- определить входные и выходные данные;
- разделить решение на понятные шаги;
- выбрать необходимые конструкции: последовательность, ветвление или цикл;
- проверить алгоритм на обычных и граничных входных значениях;
- после проверки записать алгоритм на языке программирования.

Алгоритмы – основа программирования. Любая программа – это алгоритм, записанный на языке программирования. Умение строить и анализировать алгоритмы – первый шаг к созданию собственных цифровых решений.

Правильный алгоритм должен быть понятен выбранному исполнителю, завершаться и выдавать ожидаемый результат для допустимых входных данных.

Глоссарий

Термин	Определение
Алгоритм	Точное описание последовательности действий, приводящих к решению задачи за конечное число шагов
Исполнитель	Объект, способный выполнять команды алгоритма

Система команд	Набор команд, которые исполнитель способен распознавать и выполнять
Входные данные	Исходная информация, поступающая для решения задачи
Выходные данные	Результат работы алгоритма
Последовательность	Алгоритмическая конструкция, в которой команды выполняются друг за другом в заданном порядке
Ветвление	Алгоритмическая конструкция: выбор действия в зависимости от условия
Условие	Высказывание или выражение, результат проверки которого может быть истинным или ложным
Цикл	Алгоритмическая конструкция: повторение группы действий несколько раз
Итерация	Одно выполнение повторяющейся группы команд цикла
Блок-схема	Графическое представление алгоритма с помощью геометрических фигур
Программа	Алгоритм, записанный на языке программирования

Материалы к занятию

- 15.pptx (17.1 МБ)